



E-CELL

AI & AUTOMATION

DOMAIN

T A S K 2



PROBLEM STATEMENT

Organizational documentation contains a wealth of critical operational knowledge, but searching and retrieving answers manually is slow, error-prone, and inefficient. Your task is to build an end-to-end knowledge management and semantic retrieval system using a publicly available corpus or a custom collection of technical PDFs.

The system must take raw unstructured documents, preprocess them, chunk and index text into machine-readable vector embeddings, execute semantic search queries, and finally deploy the best configuration through a clean API endpoint.

DATASET

Custom multi-document text corpus / Local directory of multi-page technical PDFs or policy manuals. Enterprise manuals and technical guides are dense, unstructured documents filled with domain-specific knowledge.

Retrieval System Setup

Using the dataset, establish a clear retrieval evaluation framework. Measurable performance metrics include:

- Contextual Groundedness (measuring and reducing LLM hallucination risk)
- Information Retrieval Depth (optimizing document token window match vs text noise)
- Any other explicit evaluation benchmark derivable from standard RAG analysis frameworks.

⚠ IMPORTANT NOTE: Your context retrieval constraints and anti-hallucination guardrails must be clearly defined and justified in your System Report. Ambiguous grounding frameworks or unhandled model hallucinations will negatively impact your evaluation score.

What You Need to Build

Your project must implement a five-stage pipeline. Each stage builds directly on the previous. Below are the first three stages in detail.

Stage 1: Document Ingestion & Text Segmentation

- Set up a pipeline to load multi-page PDFs and text files from your document corpus.
- Extract raw text strings from files while filtering out parsing layout noise.
- Strip operational data by removing unnecessary structural headers, footers, boilerplate text, and systemic artifacts.
- Isolate or parse document structural changes where possible:
 - Standard Operational Procedures
 - Corporate Policy Frameworks
 - Compliance Regulations
 - Technical Troubleshooting Logs

Stage 2: Embedding Generation & Indexing

Convert segmented text blocks into machine-readable numeric vector spaces using one or more of the following:

Local Dense Transformers

Models: all-MiniLM-L6-v2, bge-large-en-v1.5

Format: Dense mathematical vector matrix

- Lightweight, local execution footprint
- Captures core semantic sentence similarity

01

API-Based Embeddings

Models: OpenAI text-embedding-3-small, Cohere

Format: High-dimensional dense representations

- Superior semantic cross-lingual capacity
- State-of-the-art vector text alignment

02

Hybrid Search Vectors

Setup: Sparse BM25 tokens combined with dense vector arrays

Format: Multi-layered index layout

- Robust keyword-matching fallback capabilities
- High precision structural text retrieval

03

Custom Payloads (Optional)

Usage: Source tracking, page/line allocations

Setup: Title tags, chunk overlap configurations

- Any engineered metadata signal you find useful

04

Stage 3: LLM Inference & Context Orchestration

Option A (Local Llm): Implement Ollama / vLLM (Llama-3-8B-Instruct) to execute full on-premise pipeline processing, ensuring total data privacy and zero external dependency.

Option B (API Llm): Integrate Google Gemini API (gemini-1.5-flash) using its zero-cost developer tier to leverage state-of-the-art reasoning and expansive context capacity.

Stage 4: Pipeline Evaluation

Evaluate the pipeline and produce a clear performance comparison report. Required Metrics:

- CR: CONTEXT RELEVANCE
- F: FAITHFULNESS (GROUNDEDNESS)
- AR: ANSWER RELEVANCE
- L: INFERENCE LATENCY
- QR: QUERY RESOLUTION RATE

Stage 5: API Deployment

- Serialize and save your system state parameters (e.g., local vector store indices, system prompt templates, or routing configurations).
- Deploy a highly responsive FastAPI application layer.
- The endpoint must accept raw query inputs and return a securely grounded answer with verifiable source metadata.
- Swagger documentation must be auto-generated and accessible at /docs.

DEPLOYMENT ENDPOINT:

POST /query Input: { "query": "..." } Output: { "answer": "...", "confidence": 0.xx, "sources": [...] }

Minimum Requirements

- Document parser and preprocessing engine running
- Text extraction pipeline (noise removal, structural cleanup)
- Text chunking strategy (with explicit chunk sizes and token overlaps)
- Indexing configuration (at minimum a local vector database index setup)
- Operational orchestration of Local LLM, API LLM, and Reranked variants
- Performance benchmarks comparing all three distinct system paths
- Detailed evaluation report focusing on groundedness metrics
- System state saving / vector database index serialization
- Production FastAPI code architecture
- Working query interface with matching structural inputs/outputs

Deliverables

1 • Source Code

A highly organized GitHub repository featuring clean, modular, production-ready Python files. Each component stage of the pipeline should run from its own script. Avoid monolithic notebooks for final submissions.

2 • System Report

A concise written report (PDF or Markdown) covering:

- Extraction choices, chunk boundaries, and token size choices
- Core embedding configurations and vector search metrics utilized
- Pipeline evaluation tables, performance charts, and error reviews
- Selected configuration setting with direct data-driven justification

3 • Live Demo

Be ready to demonstrate the following during evaluation:

- Dynamic data ingestion and document processing using unseen PDF files
- Active text matching logs and internal vector distance scoring outputs
- Pipeline evaluation summaries showing clear metric tracing
- Live FastAPI /query endpoint resolving complex situational prompts

Suggested Folder Structure

```
project/
├── data/                # raw documents directory and local vector database indices
├── notebooks/          # exploratory text parsing and prompt architecture testing
├── src/
│   ├── preprocess.py   # document parsing, cleaning, and text chunk splitting
│   ├── features.py     # embedding generation and vector database initialization
│   ├── train.py        # pipeline retrieval query orchestration loop
│   ├── evaluate.py     # RAG evaluation metrics tracking (Faithfulness, Relevance)
│   └── utils.py        # core helper modules and text execution logs
├── api/
│   └── app.py          # FastAPI backend server configuration
├── models/             # saved pipeline parameters and serialized indices
├── README.md           # comprehensive system engineering report
└── requirements.txt    # environment dependency declarations
```



SUBMISSION GUIDELINES

DEADLINE - 21ST JUNE

Meet the deadlines, late submissions will not be entertained PDF format of the ppt must be submitted.

For any clarifications reach out to:

Buvanesh- +91 72006 27711

Aarya - +91 84689 48933

Nitesh- +91 79046 01851

Saksham- +91 92364 51691